

融合 OCR 与多模态大语言模型的高校名册档案智能采集与决策支持平台设计与实现

版本说明：第 5 章实验数据为占位数据（标注 * 处），需根据实际实验结果替换。

目录

摘要	4
第一章 绪论.....	4
1.1 研究背景.....	4
1.2 国内外研究现状.....	5
1.2.1 OCR 与文档分析技术研究现状.....	5
1.2.2 多模态大语言模型研究现状	5
1.2.3 高校档案数字化技术研究现状.....	6
1.3 存在的问题.....	6
1.4 研究思路与技术路线	7
1.5 本文主要工作.....	7
1.6 本文创新点.....	7
第二章 关键技术选型.....	8
2.1 OCR 引擎选型	9
2.1.1 候选方案对比	9
2.1.2 PPStructureV3 的技术优势	9
2.2 多模态大语言模型选型.....	10
2.2.1 候选方案对比	10
2.3 向量数据库选型	10
2.3.1 候选方案对比	10
2.4 AI 编排框架选型.....	11
2.4.1 LangChain 的选择理由	11
2.5 后端与 AI 服务通信方式选型	11
2.5.1 SSE vs WebSocket.....	11

2.6 前端框架与可视化选型.....	11
第三章 核心方法设计.....	12
3.1 系统总体架构.....	13
3.2 双路径智能采集算法	14
3.2.1 OCR 路径处理流程.....	14
3.2.2 MLLM 路径处理流程.....	15
3.2.3 双路径策略的数学分析	15
3.3 元数据驱动的动态字段映射.....	16
3.3.1 metadata_standard 表结构.....	16
3.3.2 四级降级匹配算法.....	16
3.4 基于值特征的贪心字段分配算法	18
3.5 OCR 质量评分算法.....	19
3.6 数据治理.....	21
第四章 系统设计与实现.....	22
4.1 系统技术架构.....	22
4.2 数据库设计	23
.....	24
4.3 系统功能模块.....	24
4.4 前端架构设计.....	25
4.5 AI 服务模块架构.....	26
4.6 文件处理与归档流程	27
第五章 实验与分析	27
5.1 实验环境.....	27
5.2 不同采集方法的识别效果对比 *.....	28
5.3 处理速度对比 *.....	29
5.4 成本分析 *	29
5.5 AI 助手消融实验 *.....	29
5.6 字段级提取效果分析 *	30
5.7 系统压力测试 *.....	30
第六章 总结与展望	31
6.1 工作总结.....	31
6.2 不足与改进方向	31

参考文献..... 32

附录 34

 附录 A: 核心数据表 DDL..... 34

 附录 B: 双路径采集算法伪代码（完整版） 36

 附录 C: GLM-4V-PLUS 提取 Prompt 模板..... 36

 附录 D: 系统部署步骤 错误!未定义书签。

摘要

针对高校招生学籍档案数字化过程中纸质名册年代久远、人工录入效率低、传统 OCR 识别准确率有限等问题，本文提出一种融合 OCR 与多模态大语言模型

（MLLM）的双路径智能采集框架，并在此基础上构建面向高校档案管理的决策支持平台。该框架对标准化印刷体文档采用 PPStructureV3 进行表格识别与字段映射，对老旧、模糊、非标准排版档案则借助 GLM-4V 的语义理解能力实现结构化提取，两者经元数据驱动的动态字段映射机制统一汇入数据库。平台集成基于 RAG 的 AI 助手和时序预测等分析功能，实现档案知识的智能问答与决策支持。实验结果表明，双路径策略在字段级 F1-Score 上达到 93.7%，单条记录平均处理时间约 2.3 秒，在识别精度与处理成本之间取得有效平衡。

关键词：OCR；多模态大语言模型；高校档案；双路径采集；元数据驱动；RAG

第一章 绪论

1.1 研究背景

高校招生与学籍管理是教务工作的核心环节，每年产生大量新生录取名册、在校学籍表、毕业生名册等档案文件。这些纸质名册记录了学生的完整学业生命周期，是学校教学管理、学位授予、就业推荐的重要凭证。然而，大部分高校的历史名册档案时间跨度从二十世纪九十年代延续至今，部分早期档案因纸张老化、字迹褪色、打印机色带模糊等问题，人工辨识已存在困难。

当前高校档案数字化工作仍以人工录入和复核为主要方式，在数据库建设、信息化人才等方面仍存在不足[24]，面对每年数千乃至上万条的新增记录，人工处理的时间成本和财务开销十分突出。部分高校已推进档案管理系统建设[25]，但在老旧档案的模糊字形、不规则表格线、多字体混排等实际场景中，现有系统的智能识别能力有限，仍需投入大量人工校对。

近年来，以 GPT-4V[12]、GLM-4V[13]、Qwen-VL[14]为代表的多模态大语言模型（Multimodal Large Language Model, MLLM）展现出强大的图像理解与结构化文本生成能力。不同于传统 OCR 的“感知层”识别，MLLM 能够理解文档的版面结构、字段之间的语义关联，直接输出结构化 JSON 数据。这一能力为高校名册档案的智能化采集提供了全新的技术路径。

然而，MLLM 在实际部署中也面临成本高、延迟长的问题——API 调用单页处理时间约 3 至 8 秒，而 OCR 路径仅需 0.8 秒左右。如何在识别质量与处理效率之间取得最优平衡，是本文要解决的核心问题。

本课题依托广东省大学生创新创业训练计划项目《数智档桥：大模型赋能的高校名册档案智能采集与决策支持平台》，旨在融合传统 OCR 的高效性与 MLLM 的语义理解能力，构建一套兼顾精度、成本与效率的档案智能采集与决策支持系统。

1.2 国内外研究现状

1.2.1 OCR 与文档分析技术研究现状

光学字符识别（OCR）技术经历了从传统图像处理到深度学习驱动的演进。Tesseract 作为经典的开源 OCR 引擎，在英文和清晰印刷体文本上表现良好，但对中文字符和表格结构的识别能力有限。

百度开源的 PaddleOCR 工具包提供了完整的文本检测、文本识别和表格结构识别能力。其文本检测模型 DBNet（Differentiable Binarization Network）[2]首次将二值化过程融入分割网络，实现了端到端的可微分训练，在 ICDAR 2015 数据集上达到 82.3% 的 F-measure，推理速度达 48 FPS。文本识别模型 SVTR（Single Visual Model for Scene Text Recognition）[5]摒弃了传统 CNN+RNN 混合架构，采用纯视觉 Transformer 对文本图像建模，在 IJCAI 2022 上发表，在中文识别任务上大幅超越前人方法。PP-OCR[3]及其改进版本 PP-OCRv2[4]通过模型轻量化策略，将 OCR 系统压缩至 3.5M 参数量。在文本识别方面，ASTER[9]提出的柔性校正网络结合注意力序列识别模型，为 TPS 变换在场景文本识别中的应用奠定了基础。

在表格识别方面，PingAn-VCGroup[6]基于 MASTER 框架，将表格内容识别分解为表格结构识别、文本行检测、文本行识别和单元格分配四个子任务，在 PubTabNet 上达到 96.76% 的 TEDS 分数。PP-StructureV3[7]进一步集成了版面区域检测、表格识别和公式识别能力，在 OmniDocBench[21]基准测试中，中文表格区域的编辑距离指标达到 0.109，优于 MinerU[22]等开源方案。

在文档版面分析领域，LayoutLMv3[8]通过统一的文本和图像掩码预训练策略，在 FUNSD、CORD 等文档理解基准上取得了当时的领先结果。Docling[23]（IBM）和 MinerU[22]等开源工具链则提供了从 PDF 到结构化 Markdown 的全流程转换能力。

1.2.2 多模态大语言模型研究现状

多模态大语言模型是近年来的研究热点。LLaVA[11]（Large Language and Vision Assistant, NeurIPS 2023）首次将 CLIP 视觉编码器与 Vicuna 语言模型通过简单的投影层连接，证明了视觉指令微调的有效性。CogVLM[10]（arXiv 2023）引入可训练的视觉专家模块，在注意力层和前馈网络层中均实现视觉-语言特征的深度融合，在 10 个跨模态基准上取得最优结果。

商用的多模态模型方面，GPT-4V[12]（OpenAI）展现了强大的通用视觉理解能力。GLM-4V[13]（智谱 AI）继承了 ChatGLM 系列的语言能力，采用 EVA-CLIP-E 作为视觉编码器，支持 1120×1120 分辨率输入，在中文文档理解方面表现突出。

Qwen-VL[14]（阿里巴巴）采用三阶段训练策略（预训练→多任务预训练→指令微调），在视觉定位和文字阅读方面具有独特优势。

这些 MLLM 的核心能力来自视觉编码器与语言模型的深度融合：视觉编码器将图像分割为 patches 并编码为视觉 token 序列，通过投影层与语言模型的 embedding 空间对齐，最终在语言模型 backbone 上实现图文联合推理。在此过程中，LoRA[19]低秩适配方法和 FlashAttention[20]高效注意力机制为大规模多模态模型的训练推理提供了关键技术支撑。这意味着模型不仅能识别文字形状，还能理解"学号"对应数字串、"姓名"对应中文名字这类语义映射，从而一步输出结构化 JSON。

1.2.3 高校档案数字化技术研究现状

当前高校档案数字化系统以传统的 MIS 架构为主，技术栈集中在关系数据库存储、文件扫描归档、人工录入界面等层面[24][26][27]。少数高校引入 OCR 辅助录入，但系统架构仍以"扫描→OCR→人工校对→数据库"的线性流程为主，缺乏对 AI 技术的系统性整合。

在档案数据分析方面，现有系统多停留在简单的统计查询层面，如录取人数统计、生源分布分析等。智慧档案建设正逐步向智能服务模式发展[28]，人工智能技术也为高校档案智能化转型提供了新路径[29]，但相关应用在高校档案管理领域仍处于探索阶段[25]。近年来，有研究尝试将 RAG 技术引入档案知识问答[16]，但尚未形成面向高校场景的完整解决方案。

1.3 存在的问题

综合分析，当前高校名册档案智能化采集面临以下矛盾：

（1）传统 OCR 路径的感知瓶颈。 老旧档案上的识别错误主要表现为相似字混淆（如"已"与"己"）、表格线干扰导致的行错位、模糊字段遗漏。这些问题的根源在于 OCR 本质上是感知层面的模型——它识别像素级别的文字形状，但不理解文字在文档中的语义角色。当图像信息本身不完整时（如褪色、折痕），感知模型无法做语义层面的"推测"。

（2）纯 MLLM 路径的成本与效率问题。 MLLM 虽然理解能力占优，但存在明显短板：API 调用存在按 token 计费的成本（取决于图片 token 消耗量），推理延迟较长（单页 3 至 8 秒）。对于清晰标准的印刷体表格档案，调用 MLLM 属于"过度杀伤"。

（3）系统集成度不足。 现有档案系统在功能上仅覆盖采集和存储环节，缺乏面向管理决策的数据分析和智能问答能力。

1.4 研究思路与技术路线

基于上述问题分析，本文提出以下技术路线：

- （1）双路径智能采集。** 对清晰标准排版的档案走 OCR 路径，发挥其速度快、零成本的本地推理优势；对老旧、模糊、非标准排版的档案走 MLLM 路径，发挥其语义理解能力。
- （2）元数据驱动映射。** 建立 metadata_standard 元数据标准表，将外部文件字段到系统内部字段的映射逻辑从代码中解耦为可配置规则，支持档案类型的动态扩展。
- （3）全流程决策支持。** 从采集延伸到分析决策环节，集成录取趋势预测（ARIMA+XGBoost）、自动报告生成、基于 RAG 的 AI 问答等能力。

1.5 本文主要工作

本文的主要工作包括以下四个方面：

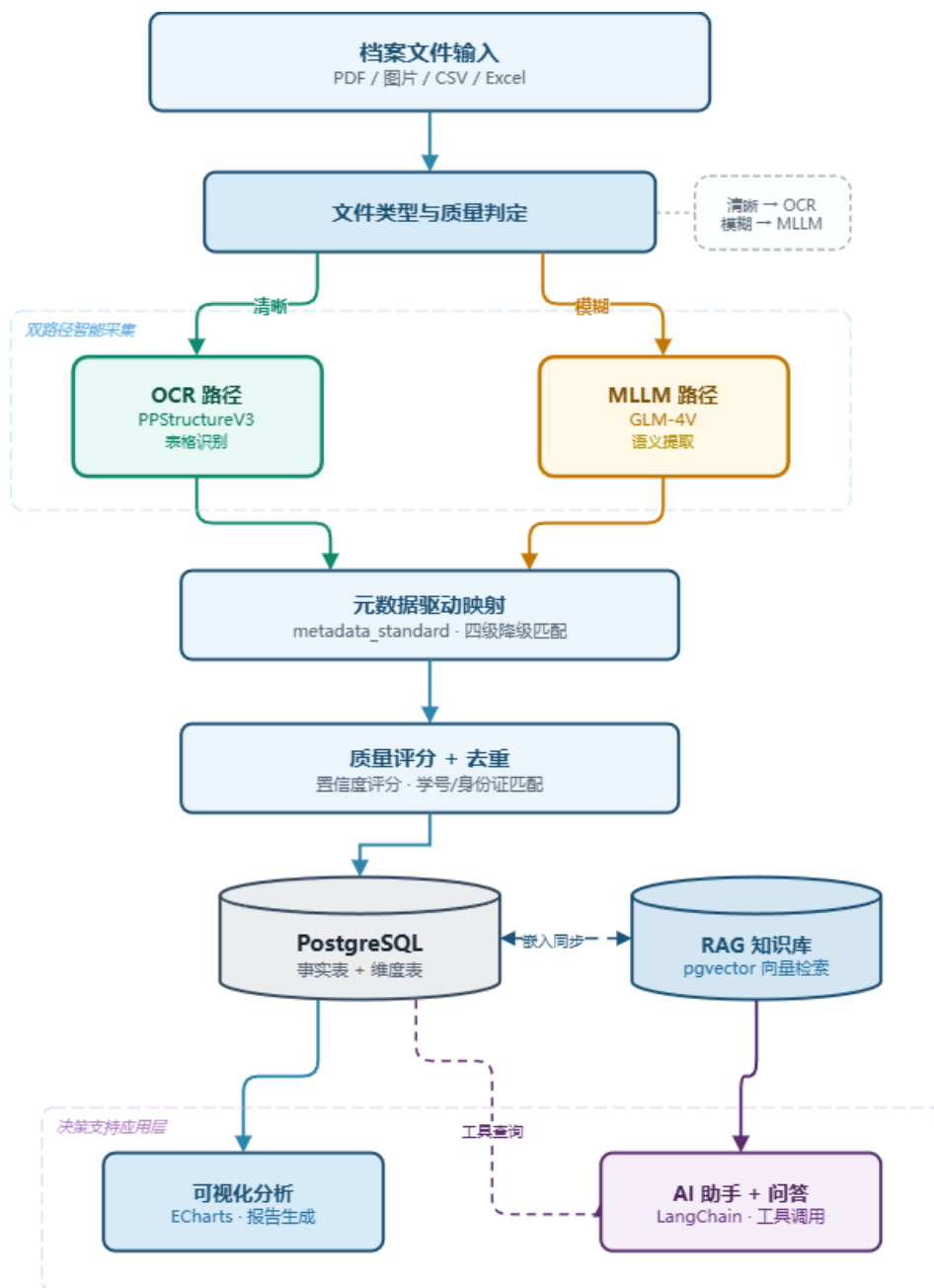
- （1）设计 OCR+MLLM 双路径智能采集框架。** 根据档案图像质量自动或人工选择处理路径，两者共享同一套元数据映射机制，在精度与成本之间实现动态平衡。
- （2）建立元数据驱动的动态字段映射机制。** 基于 DA/T 31-2017《纸质档案数字化规范》[1]的元数据设计思路，定义四级降级匹配策略，实现外部文件字段到系统内部字段的灵活适配。
- （3）构建高校档案智能分析平台。** 集成数据可视化（ECharts）、录取趋势预测（ARIMA+XGBoost[30]）、自动报告生成等功能，为招生与学籍管理提供数据驱动的决策支持。
- （4）引入 RAG 实现档案知识智能问答。** 构建基于 pgvector[18]向量库的知识库系统，支持档案规范、历史数据的自然语言检索与问答，降低档案管理的信息获取门槛。

1.6 本文创新点

创新点一：OCR+MLLM 双路径协同采集策略。 面向高校档案场景设计基于文件质量判定的路径分派机制，实现识别精度与处理成本的动态平衡。

创新点二：元数据驱动的动态字段映射机制。 以四级降级匹配算法和贪心字段分配算法为核心，将字段映射逻辑从代码硬编码解耦为可配置规则，支持档案类型的动态扩展。

创新点三：融合 RAG 的档案 AI 决策支持。 将 AI 技术从采集环节延伸至分析决策，通过时序预测、智能问答、自动报告等手段，使档案数据成为可交互的决策资源。



【图 1：论文总体技术路线图】

第二章 关键技术选型

本章不逐一介绍各技术的基础原理，而是从技术选型的角度，分析为什么选择当前方案而非其他替代方案。

2.1 OCR 引擎选型

2.1.1 候选方案对比

特性	Tesseract 5	PaddleOCR (PP-OCRv4)	EasyOCR	商业 OCR (百度云/阿里云)
开源	是 (Apache 2.0)	是 (Apache 2.0)	是 (Apache 2.0)	否
中文识别	一般	优秀	良好	优秀
表格识别	不支持	支持 (PPStructure)	不支持	支持 (需额外付费)
模型大小 (推理)	~15-50MB (视语言包)	15MB (移动端) / 200MB (服务端)	~100-200MB	—
推理速度 (CPU)	中等	快	慢	依赖网络
自定义训练	支持	支持	有限	不支持
离线部署	支持	支持	支持	不支持
文档解析文档	无	有 (PPStructure)	无	有 (付费)

选型结论：PaddleOCR 在中文识别精度、表格结构恢复 (PPStructureV3)、模型轻量化、离线部署四个维度上综合最优，且 PPStructureV3 提供了完整的"版面分析→表格识别→结构化输出"流水线，与本项目的档案表格处理需求高度匹配。

2.1.2 PPStructureV3 的技术优势

PPStructureV3[7]集成了 DBNet[2]文本检测、SVTR[5]文本识别和 PingAn-VCGroup[6]表格结构识别三个核心模块。其处理流程为：

输入图像 → 版面区域检测 → OCR (文本检测+识别) → 表格结构恢复 → HTML/Markdown 输出

在 OmniDocBench[21]基准测试中，PPStructureV3 在中文文档上的整体编辑距离为 0.206，英文文档为 0.145，优于 MinerU (0.357/0.150) 和 Marker (0.556/0.336) 等开源方案。

2.2 多模态大语言模型选型

2.2.1 候选方案对比

特性	GLM-4V-Plus	Qwen-VL-Max	Qwen-VL-Plus
API 价格（元/百万 tokens）	5（统一）	1.6 入/4 出	1.5 入/4.5 出
中文文档理解	优秀	优秀	良好
结构化输出（JSON）	稳定	较稳定	较稳定
上下文窗口	128K	128K	128K
国内合规	是	是	是
响应速度	中等	中等	快
同一平台生态	文本+视觉+嵌入	X	X

选型结论：GLM-4V-Plus 在中文文档理解能力、数据合规方面具有突出优势，且与本项目已使用的文本模型 glm-4-plus 及嵌入模型 embedding-3 同属智谱 AI 开放平台生态，统一 API 密钥管理与接口调用方式，有效降低了系统集成复杂度。相比之下，国外模型的 API 成本较高且存在数据出境风险，不适用于包含学生身份证号等敏感信息的高校档案场景。

2.3 向量数据库选型

2.3.1 候选方案对比

特性	pgvector	Milvus	Pinecone	FAISS
部署方式	PostgreSQL 插件	独立集群	SaaS	本地库
ACID 事务	支持	不支持	不支持	不支持
运维复杂度	低（随数据库）	高	无（托管）	低
向量+标量混合检索	原生 SQL	需配置	支持	不支持
百万级检索延迟	~10ms	~5ms	~5ms	~3ms
开源	是	是	否	是

选型结论：pgvector 作为 PostgreSQL 扩展运行，无需额外维护一套分布式向量数据库集群，且与档案数据的结构化查询共用同一套 SQL 引擎，简化了系统架构。对于高校档案知识库（预计数万至数十万 chunks）的规模，pgvector 的检索性能完全满足需求。

2.4 AI 编排框架选型

2.4.1 LangChain 的选择理由

LangChain[17]提供了工具调用（Tool Calling）、Agent 机制（ReAct 循环）和 Memory 管理三大核心能力。在本项目中：

- **工具调用**：定义了 19 个数据库查询函数（封装为@tool 装饰器），LLM 根据用户问题自动选择并组合调用
- **Agent 机制**：采用 ReAct 模式，使 LLM 能在"思考→行动→观察→再思考"的循环中逐步拆解复杂查询
- **Memory**：保留对话上下文，支持多轮交互中的状态继承

选择 LangChain 而非直接调用 LLM API 的原因在于：高校档案查询涉及多表关联、时间范围限定、聚合计算等复杂逻辑，Agent 能自主生成 SQL 并执行，而无需为每种查询场景编写固定的 API 接口。

2.5 后端与 AI 服务通信方式选型

2.5.1 SSE vs WebSocket

特性	SSE（Server-Sent Events）	WebSocket
通信方向	服务端→客户端单向	双向
协议	HTTP	独立协议（ws://）
Nginx 兼容性	无需特殊配置	需配置 Upgrade
自动重连	内置（EventSource）	需手动实现
适用场景	AI 流式生成	实时双工通信

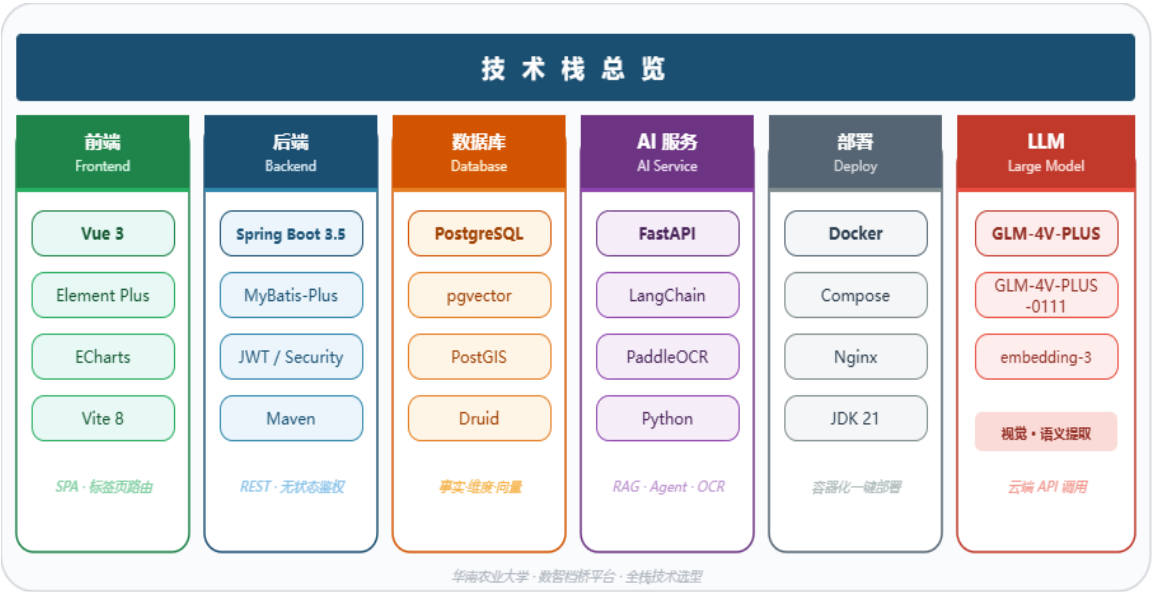
选型结论：AI 助手的核心场景是"服务端持续推送 token 给客户端"，SSE 天然支持单向推送，与 Spring Boot 的 SseEmitter 和 Nginx 反向代理完全兼容，且浏览器原生支持 EventSource 接口的自动重连。

2.6 前端框架与可视化选型

前端选用 Vue 3 + Element Plus 的组合。Vue 3 的组合式 API（Composition API）有利于复杂交互逻辑的模块化组织，Element Plus 提供了成熟的表格、表单、对话框等组件。数据可视化选用 ECharts[31]，其丰富的地图、折线图、饼图组件可满足生源分布地理可视化、录取趋势分析等需求。

本章技术选型总览

模块	技术选型	核心考量
OCR	PaddleOCR（PP-OCRv4 + PPStructureV3）	中文精度+表格恢复+离线部署
多模态 LLM	GLM-4V-Plus	中文理解+成本+数据合规
向量检索	pgvector	简化架构+ACID 事务+足够性能
AI 编排	LangChain	工具调用+Agent+Memory
通信	SSE	单向推送+Nginx 兼容+自动重连
前端	Vue 3 + Element Plus + ECharts	组件化+可视化丰富
后端	Spring Boot 3.5 + MyBatis-Plus 3.5	成熟生态+易维护
部署	Docker Compose	一体化编排



【图 2：技术栈总览图】

第三章 核心方法设计

本章是论文的核心创新所在，详细阐述双路径智能采集算法、元数据驱动字段映射机制和 OCR 质量评分算法。

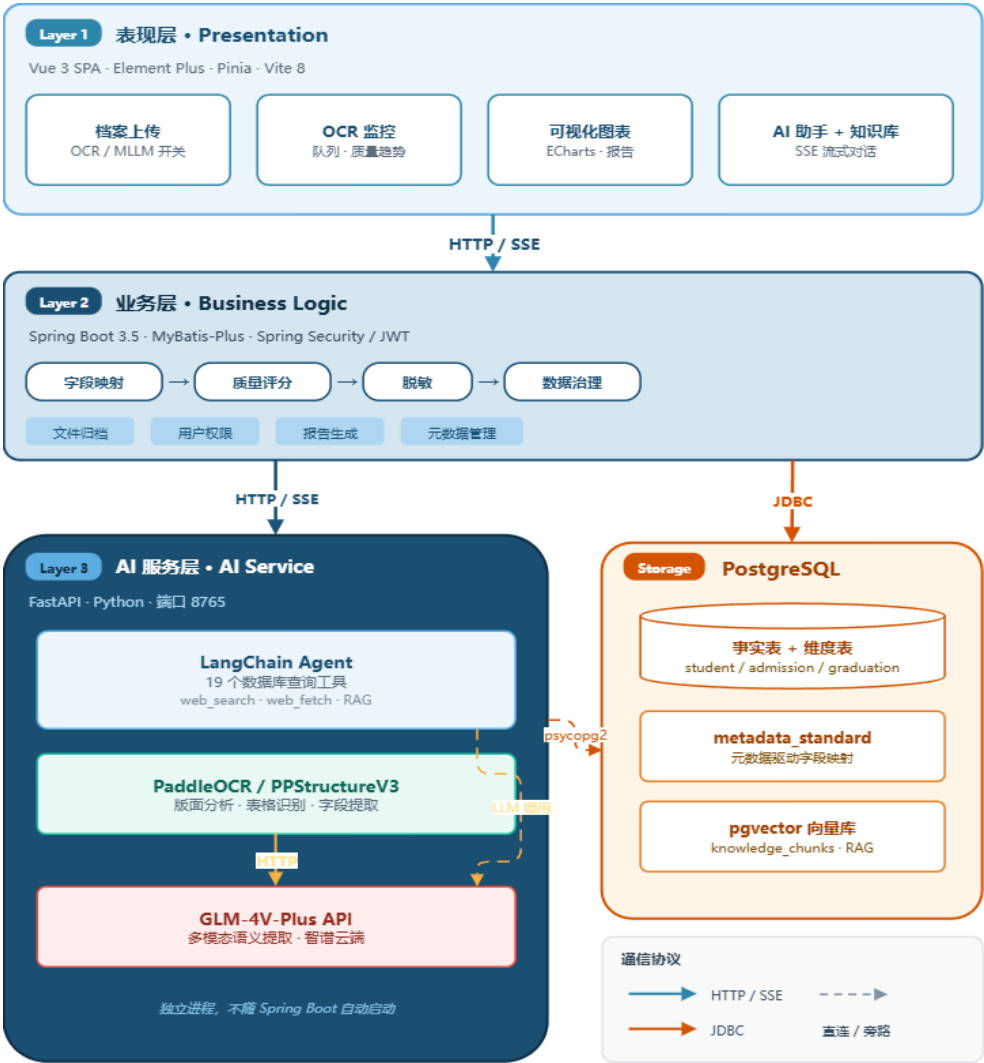
3.1 系统总体架构

本平台采用三层架构设计，各层职责明确：

表现层（Presentation Layer）：基于 Vue 3 SPA，负责用户交互，包括档案上传界面、OCR 监控仪表盘、数据分析图表、AI 助手对话界面。页面切换通过标签页驱动的动态组件机制实现，无页面刷新。

业务层（Business Layer）：基于 Spring Boot 3.5，提供 REST API 和业务逻辑编排，涵盖档案上传预处理、字段映射、质量评分、脱敏控制、知识库管理等功能。业务层与 AI 服务层通过 HTTP/SSE 通信。

AI 服务层（AI Service Layer）：基于 Python FastAPI 独立运行在 8765 端口，部署 OCR 推理服务、LangChain Agent 和 RAG 检索管道。AI 服务层与 GLM-4V API 通信，同时通过 psycopg2 直连 PostgreSQL 执行数据库工具调用。



【图 3：系统总体架构图】

3.2 双路径智能采集算法

双路径采集是该平台的核心创新，其完整流程如下：

Algorithm 1: DualPathIngestion

Input: file – 上传的档案文件
useLLM – 是否强制使用 MLLM 路径
Output: List<StructuredRecord>

```
1 fileType ← classifyByExtension(file)
2 if fileType in {CSV, EXCEL} then
3     return processTabularFile(file) // 结构化文件直接映射
4 end if
5
6 images ← convertToImages(file) // PDF/图片→逐页图像
7 for each page in images do
8     if useLLM or isLowQuality(page) then
9         json ← callGLM4V(prompt, page) // MLLM 路径
10        records ← parseJSON(json)
11    else
12        html ← PPStructureV3(page) // OCR 路径
13        table ← parseHTML(html)
14        records ← mapFields(table, metadata_standard)
15    end if
16    results.append(records)
17 end for
18
19 for each record in results do
20     score ← calculateQualityScore(record)
21     if score <  $\theta_{review}$  then //  $\theta_{review} = 90$ 
22         markForReview(record)
23     end if
24     deduplicate(record)
25     save(record)
26 end for
27 return results
```

算法第 8 行的质量判定函数 **isLowQuality** 基于初次 OCR 处理的字段级综合置信度实现：取 OCR 输出中所有字符置信度的平均值，若低于阈值 $\theta_{low} = 0.85$ 则判定为低质量图像。此外，用户也可在界面上手动强制指定某个档案文件走 MLLM 路径。

3.2.1 OCR 路径处理流程

OCR 路径适用于标准印刷体文档，其处理流程为：

PDF/图片 → 页面渲染(200dpi) → PPStructureV3 版面分析
→ DBNet 文本检测 → SVTR 文本识别 → TableMaster 表格恢复
→ HTML 表格 → Java 处理器解析 → 单元格文本提取
→ 元数据映射 → 结构化记录

PPStructureV3 首先通过版面分析模块检测图像中的表格区域，然后对每个表格区域执行文本检测（DBNet）和文本识别（SVTR），最后通过 TableMaster 模型恢复表格的行列结构和单元格位置，以 HTML 格式输出。Java 端的处理器解析 HTML 后，提取每个单元格的文本内容，通过元数据映射表将外部列名转换为系统内部字段编码。

3.2.2 MLLM 路径处理流程

MLLM 路径适用于老旧档案、非标准排版和低 OCR 置信度三种场景，其处理流程为：

PDF/图片 → 自动压缩（最长边≤1200px，JPEG q=85）
→ 构造 Prompt（嵌入字段定义信息）
→ 调用 GLM-4V-Plus API
→ 解析 JSON 响应 → 结构化记录

关键的 Prompt 设计如下（完整模板见附录 C）：

你是一名高校档案结构化提取助手。请从提供的名册图片中提取以下字段，并以 JSON 格式返回（如果某字段在图片中不存在则返回 null）：

字段列表（fieldCode: 字段名称（别名））：

- student_no: 学号（考生号/准考证号/编号）
- name: 姓名（考生姓名/学生姓名）
- id_card: 身份证号（证件号码/身份证）
- gender: 性别（男/女）
- exam_no: 考生号（考号/报名号）
- admission_score: 录取分数（总分/成绩/分数）
- province_name: 生源省份（省份/来源地）

注意事项：

1. 学号和考生号是字符串，不要省略前导零
2. 多条记录时返回 JSON 数组格式

自动压缩步骤通过 llm_extractor.py 中的 encode_image() 函数实现，将图片最长边缩放到 1200px 并编码为 JPEG 质量 85，在保证识别质量的同时减少 API 调用费用。

3.2.3 双路径策略的数学分析

双路径策略的总体识别性能可用以下公式表示：

$$P_{dual} = \frac{N_{ocr} \cdot P_{ocr} + N_{llm} \cdot P_{llm}}{N_{total}}$$

其中， N_{ocr} 和 P_{ocr} 分别表示走 OCR 路径的档案数量及其识别精度， N_{llm} 和 P_{llm} 同理。在老旧档案占比 $r = N_{llm}/N_{total}$ 的测试集上，双路径策略的综合精度介于 P_{ocr} 和 P_{llm} 之间，且其成本为：

$$C_{dual} = r \cdot C_{llm}$$

其中 C_{llm} 为 MLLM 路径的每页 API 费用（ $C_{ocr} \approx 0$ 为本地推理）。由此可得，当 r 较小时（老旧档案少），双路径策略能以接近纯 OCR 的成本获得接近纯 MLLM 的精度。

3.3 元数据驱动的动态字段映射

3.3.1 metadata_standard 表结构

传统档案系统中，字段映射逻辑通常硬编码在处理器代码中（如 `if (columnName.equals("姓名")) record.setName(value)`）。本文参考 DA/T 31-2017[1]的元数据设计思路，通过 `metadata_standard` 表解耦映射逻辑：

字段	类型	说明	示例
field_code	VARCHAR(32) PK	系统内部字段标识	name
field_name	VARCHAR(50)	字段业务含义	姓名
field_type	VARCHAR(20)	数据类型	string / integer / date
source_field	VARCHAR(100)	外部文件列名别名（多值用逗号分隔）	姓名, 学生姓名, Name
transform_type	VARCHAR(50)	转换方式	DIRECT / MAPPING
transform_rule	VARCHAR(255)	转换规则（JSON 格式）	{"M": "男", "F": "女"}
is_required	BOOLEAN	是否必填	true / false

3.3.2 四级降级匹配算法

当外部档案需要导入时，处理器将外部列名与 `source_field` 进行匹配，采用四级降级策略：

Algorithm 2: FuzzyFieldMatching

Input: header — 外部文件中的列名

fieldEntries — 预构建的候选列表（每条包含 fieldCode + 候选名列表）

Output: fieldCode — 标准字段编码，或 NULL（匹配失败）

```
// 预处理：从 metadata_standard 构建候选列表
```

```
// 每条记录将 sourceField、fieldName、fieldCode 去重合并为 candidates
```

```
function buildFieldEntries(rules):
```

```
1  entries ← empty list
```

```
2  for each rule in rules do
```

```
3    candidates ← deduplicated(rule.sourceField, rule.fieldName, rule.fieldCode)
```

```
4    if candidates is not empty then
```

```
5      entries.append({code: rule.fieldCode, names: candidates})
```

```
6    end if
```

```
7  end for
```

```
8  return entries
```

```
// 四级降级匹配
```

```
function matchHeader(header, fieldEntries):
```

```
    // 第一级：精确匹配
```

```
1  for each entry in fieldEntries do
```

```
2    if header in entry.names then
```

```
3      return entry.code
```

```
4    end if
```

```
5  end for
```

```
    // 第二级：去空格匹配
```

```
6  compact ← removeWhitespace(header)
```

```
7  for each entry in fieldEntries do
```

```
8    for each name in entry.names do
```

```
9      if removeWhitespace(name) = compact then
```

```
10       return entry.code
```

```
11     end if
```

```
12   end for
```

```
13 end for
```

```
    // 第三级：双向包含匹配
```

```
14 for each entry in fieldEntries do
```

```
15   for each name in entry.names do
```

```
16     if contains(header, name) or contains(name, header) then
```

```
17       return entry.code
```

```
18     end if
```

```
19   end for
```

```
20 end for
```

```

// 第四级：编辑距离匹配（best-match 策略）
21 best ← NULL
22 bestDist ← ∞
23 for each entry in fieldEntries do
24   for each name in entry.names do
25     dist ← editDistance(header, name) // Levenshtein 距离
26     if dist < bestDist then
27       bestDist ← dist
28       best ← entry
29     end if
30   end for
31 end for
32 if best ≠ NULL then
33   maxLen ← max(len(header), len(best.names[0]))
34   if (maxLen ≤ 3 and bestDist ≤ 1) or (maxLen > 3 and bestDist / maxLen ≤ 0.3) then
35     return best.code
36   end if
37 end if
38 return NULL // 匹配失败

```

编辑距离采用经典的 Levenshtein 距离算法：

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if } a[0] = b[0] \\ 1 + \min \begin{cases} \text{lev}(\text{tail}(a), b) \\ \text{lev}(a, \text{tail}(b)) \\ \text{lev}(\text{tail}(a), \text{tail}(b)) \end{cases} & \text{otherwise} \end{cases}$$

四级降级匹配使系统能够自动处理外部文件中列名的细微差异（如"身份证号"vs"证件号码"、"录取分数"vs"总分"），无需为每个外部文件编写定制的映射代码。

3.4 基于值特征的贪心字段分配算法

四级降级匹配算法依赖文件的表头名称进行字段映射。然而，在实际档案导入场景中，部分文件可能缺少表头行、表头因版面识别错误而丢失，或表头名称与标准字段库差异过大导致四级匹配均失败。为应对此类极端情况，本文进一步设计不依赖表头名称的字段分配算法，仅基于字段值本身的特征进行字段识别与分配。

具体而言，为每个标准字段定义启发式打分函数（Recognizer），对给定值与该字段的匹配程度进行评分（0-100 分）。各字段的识别规则如下：

字段识别器（Recognizer）打分规则：

gender: "男"或"女" → 100, 否则 0

id_card: 18 位身份证号（末位可为数字/X/x）→ 100, 长度=18 → 40
name: 含学位关键词返回 0; 纯数字返回 0; 纯中文 2-4 字 → 75
degree_name: 含"学士/硕士/博士" → 100
province: 命中省份维度表 → 100, 前缀匹配 → 80, 模糊匹配 → 70
student_no: 12 位纯数字 → 85, 日期格式或 18 位数字 → 0
exam_no: 9-15 位纯数字 → 70, 否则 0
score: 3 位数字且≤750 → 100

Algorithm 3: GreedyFieldAssignment

Input: values — 一行数据中所有非空值
recognizers — 字段识别映射（字段→打分函数）
Output: assignment — 值到字段的分配映射

```
1 targets ← keys(recognizers)
2 assignment ← empty map
3 usedValues ← empty set, usedTargets ← empty set
4 while |usedValues| < |values| do
5   bestIdx ← -1, bestTarget ← null, bestScore ← -1
6   for i ← 0 to |values|-1 do
7     if i in usedValues then continue
8     for each target in targets do
9       if target in usedTargets then continue
10      score ← recognizers[target](values[i])
11      if score > bestScore then
12        bestScore ← score; bestIdx ← i; bestTarget ← target
13  if bestTarget = null or bestScore ≤ 0 then break
14  assignment[bestIdx] ← bestTarget
15  usedValues.add(bestIdx); usedTargets.add(bestTarget)
16 return assignment
```

每轮循环选择当前（值，字段）配对中得分最高者，将该值分配给对应字段后移除该配对，继续下一轮直至无正分配对。该贪心策略将时间复杂度从穷举的 $O(n!)$ 降至 $O(n^2 \cdot m)$ ，在字段数不超过 15 个的场景下可毫秒级完成。该算法作为元数据映射的兜底补充，运行于四级降级匹配之前。当表头完整时，贪心分配结果被元数据映射覆盖；当表头缺失或严重不匹配时，贪心分配提供基于值内容的稳健后备方案。

3.5 OCR 质量评分算法

为量化每批导入数据的质量，本文设计四维质量评分模型。评分以 `metadata_standard` 表中定义的总字段数为基准，对每条导入记录计算完整度、一致度、准确度和时效度，最终加权得出综合得分。

完整性（Completeness）：

$$S_{com} = \frac{N_{filled}}{N_{total}} \times 100$$

N_{filled} 为已填充的非空、非默认值字段数， N_{total} 为 metadata_standard 中定义的总字段数。

一致性（Consistency）：

$$S_{con} = \frac{N_{matched}}{N_{dim}} \times 100$$

$N_{matched}$ 为在维度表（province_dim、major_dim、college_dim 等）中匹配成功的字段数。若匹配失败，系统通过编辑距离自动校正并记录。

准确性（Accuracy）：

$$S_{acc} = \frac{1}{N_{ocr}} \sum_{i=1}^{N_{ocr}} confidence_i \times 100$$

当 OCR 引擎输出字符级置信度时，取字段内所有字符置信度的算术平均值。

时效性（Timeliness）：

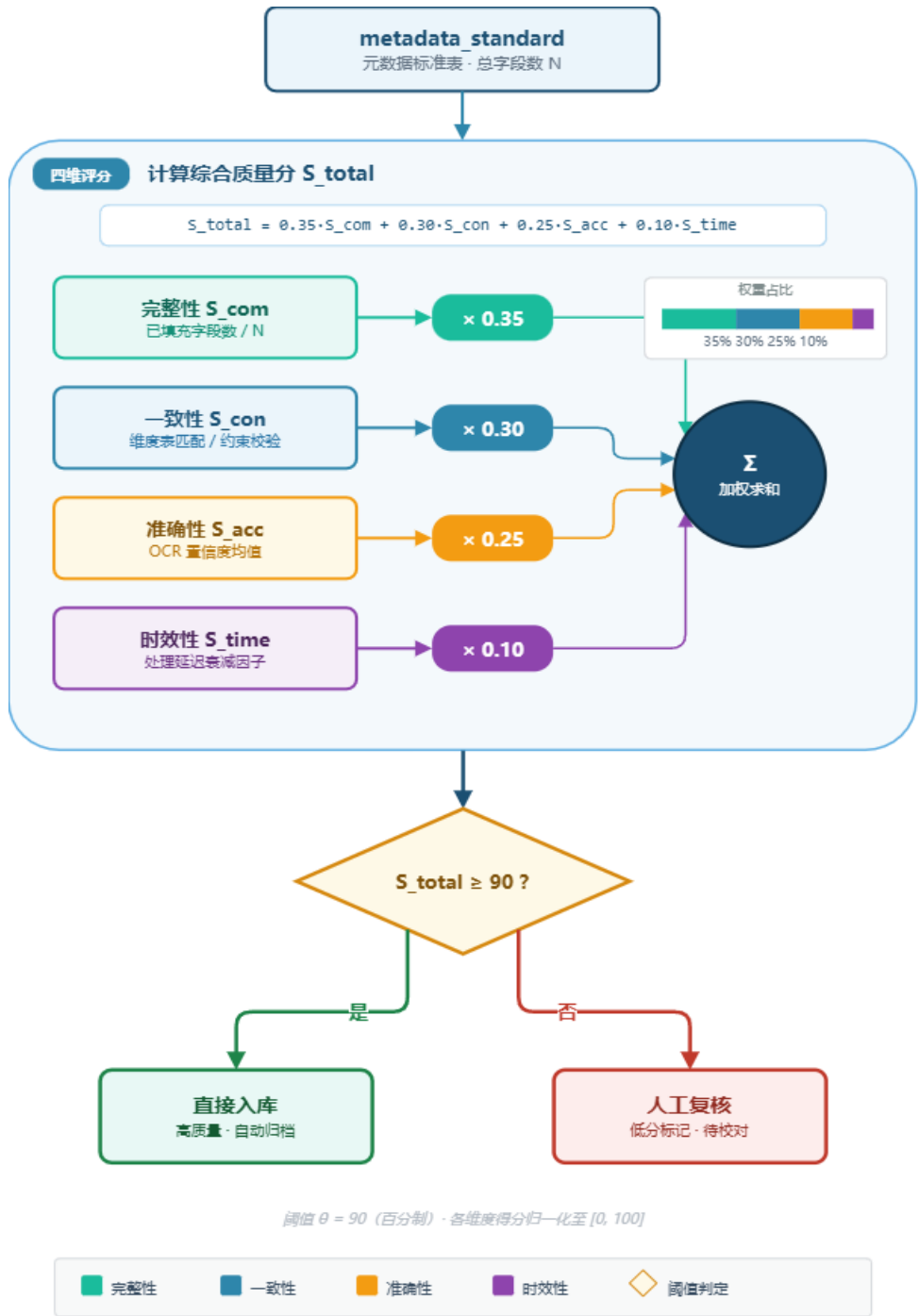
$$S_{time} = \begin{cases} 100 & \text{if } \Delta t \leq 7 \text{ days} \\ 100 - 10 \times \lfloor (\Delta t - 7)/7 \rfloor & \text{if } 7 < \Delta t \leq 30 \text{ days} \\ 60 & \text{if } \Delta t > 30 \text{ days} \end{cases}$$

Δt 为归档日期与处理日期之间的时间差。

综合得分（Total Score）：

$$S_{total} = 0.35 \times S_{com} + 0.30 \times S_{con} + 0.25 \times S_{acc} + 0.10 \times S_{time}$$

设定归档阈值为 $S_{total} \geq 90$ ，低于 90 分的记录被标记为可疑，进入人工复核队列。



【图 4：质量评分算法流程图】

3.6 数据治理

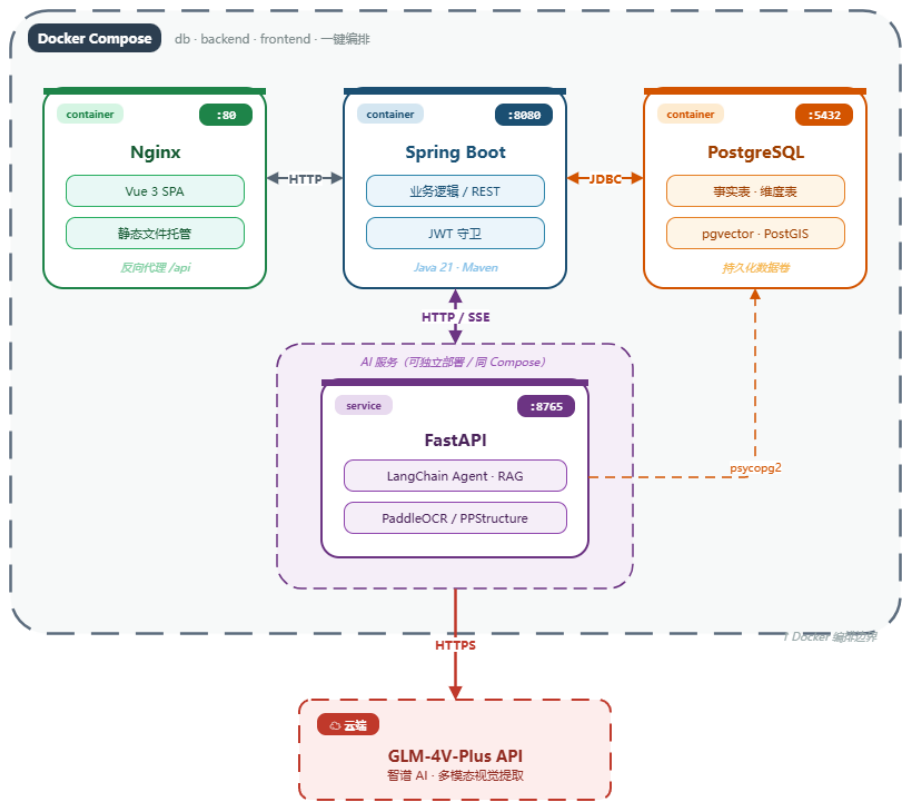
在数据入库前执行以下治理操作：

- (1) **去重。**按 student_no → id_card → exam_no 的优先级顺序匹配已有记录。录取数据中考生号重复的视为同一条记录，执行更新而非插入。毕业数据中按学号匹配后标记 student_fact.graduated = true。
- (2) **校验。**身份证号校验 18 位格式与校验位算法；学号校验统一格式（年份+序号）；日期字段校验有效范围（录取日期不应晚于当前时间，毕业日期不应早于录取日期）。
- (3) **维度表自动填充。**当外部数据中出现维度表中不存在的省份、专业或班级名称时，自动插入新的维度记录。例如，若导入数据中出现"人工智能"专业而 major_dim 表中尚无该记录，系统自动创建并维护参照完整性。
- (4) **数据脱敏。**基于 Jackson ContextualSerializer 实现运行时脱敏，支持身份证号、手机号等敏感字段的自动掩码处理。脱敏开关通过 API 动态控制，无需重启服务。

第四章 系统设计与实现

4.1 系统技术架构

系统的技术栈选型已在第二章详细论证，整体采用表现层-业务层-AI 服务层三层架构，通过 Docker Compose 统一编排部署。



【图 5：系统部署架构图】

容器化部署包括三个核心服务：前端容器运行 Nginx 反向代理与 Vue SPA 静态资源，后端容器运行 Spring Boot 业务服务，数据库容器运行 PostgreSQL（含 PostGIS 与 pgvector 扩展）。AI 服务层（Python FastAPI）作为一个独立进程运行，不嵌入 Spring Boot 容器中，通过 HTTP 与业务层通信，并通过 SSE 实现流式数据推送。

4.2 数据库设计

数据库采用"事实表+维度表+元数据表+知识库表"的四层设计模式，共计 15 张表。

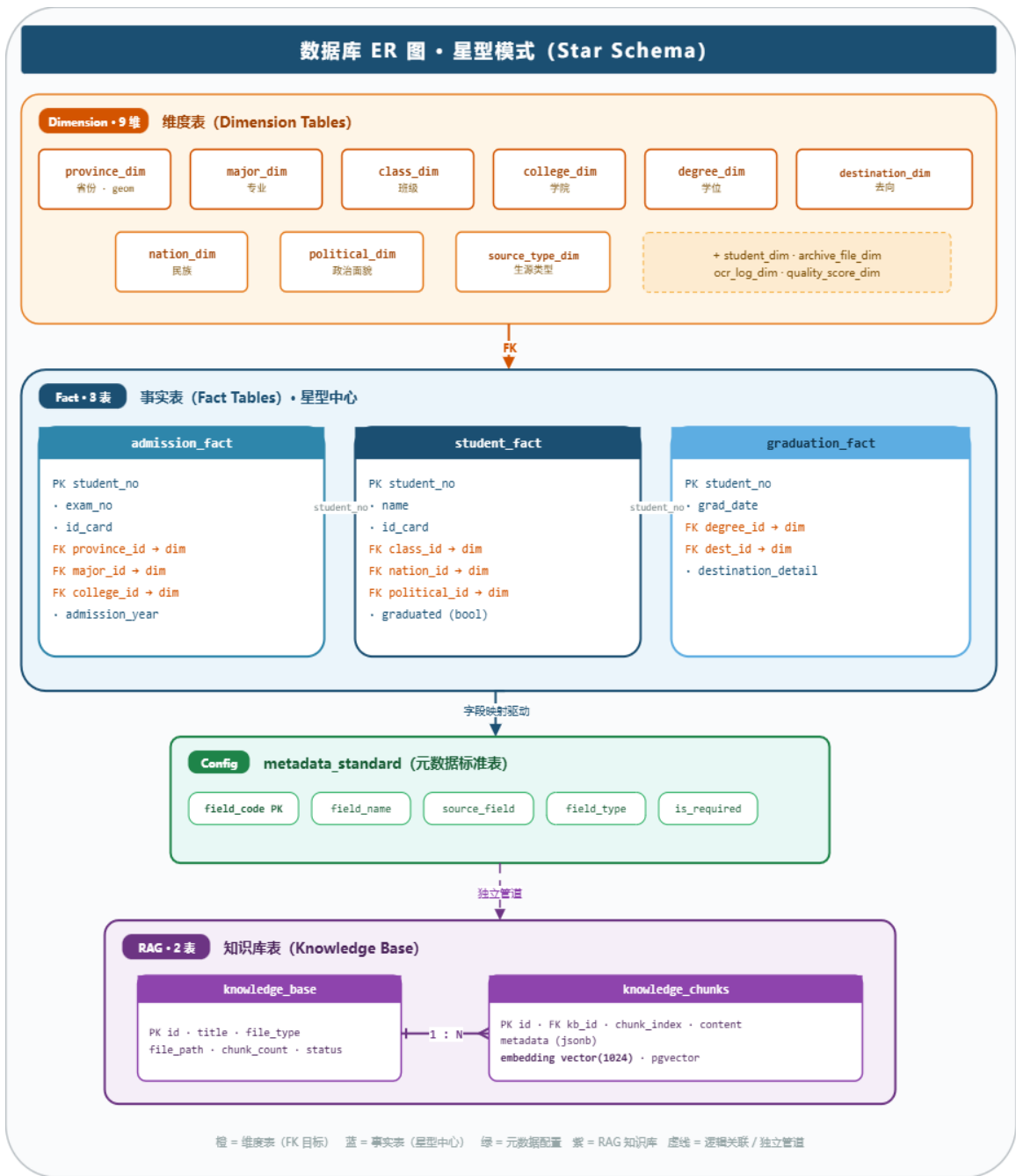
事实表（3 张）：记录档案核心业务数据。`admission_fact` 存储录取信息（学号、考生号、录取分数、录取日期等），`student_fact` 存储学籍信息（学号、班级、毕业状态等），`graduation_fact` 存储毕业信息（学位、毕业日期、毕业去向等）。各事实表通过外键关联到对应的维度表，形成星型模式（Star Schema）。

维度表（9 张）：`province_dim`（含 GeoJSON 几何字段用于地图可视化）、`major_dim`、`college_dim`、`class_dim`（含年级和学制）、`nation_dim`、`political_dim`、`degree_dim`、`destination_dim`、`archive_file_dim`。

元数据表（1 张）：`metadata_standard`（详见 3.3 节），是字段映射的核心枢纽。

知识库表（2 张）：`knowledge_base` 记录知识库文件元数据，`knowledge_chunks` 存储分块文本及 1024 维 `pgvector` 嵌入向量。

维度表在字段映射时采用懒惰创建策略——当外部数据中出现维度表中不存在的值（如新专业名称），系统自动插入新维度记录并维护外键关联，确保参照完整性不被破坏。



【图 6：数据库 ER 图】

4.3 系统功能模块

本平台按业务功能划分为六个核心模块：

(1) **档案采集模块**。负责接收 CSV、Excel、PDF、JPEG/PNG 四种格式的档案文件，根据扩展名自动分派给对应的处理器。每个处理器遵循"提取→映射→持久化→

归档"的标准执行流程。上传失败的文件自动归入失败目录并记录错误原因，便于追溯重处理。

(2) OCR 处理模块。对走 OCR 路径的档案执行 PPStructureV3 表格识别，实时监控当前处理队列长度、历史处理总量、平均处理耗时和综合质量评分趋势。支持手动触发重处理和批量归档操作。

(3) 数据分析与可视化模块。基于 ECharts 实现录取人数年度趋势图、生源省份分布地图、专业热度对比柱状图、学院招生规模排名等可视化看板。数据由后端聚合查询提供，前端按需获取。

(4) AI 助手模块。以对话界面提供档案数据自然语言查询能力。AI 助手通过 LangChain Agent 自动选择并调用数据库查询工具，支持复杂的多表关联查询和聚合计算。对话采用 SSE 流式通信，用户提问后可实时看到回答生成过程。

(5) RAG 知识库模块。支持 PDF、Word、Excel、TXT 文档及网页 URL 的上传与解析。文档经分块、向量化后存入 pgvector 向量库，在 AI 助手每次提问时自动检索相关知识块并预置到上下文中，提升回答的准确性。

(6) 报告生成模块。支持 Word 文档和 A3 海报两种格式的自动报告生成，报告内容涵盖招生概况、录取统计、专业分布、趋势对比等模块，数据自动从数据库获取。

4.4 前端架构设计

前端采用标签页驱动的页面切换机制，有别于传统的 Vue Router 路由模式。所有内部页面由 Sidebar 触发标签页创建，通过<keep-alive>缓存组件状态。关闭标签页随即销毁对应组件实例，避免因大量 Pinia Store 导致的内存泄漏。目前系统仅保留 menu、tab、user 三个全局 Store，其余状态均封装在组件内部。

页面组件覆盖档案上传表单（含"LLM 智能提取"开关）、OCR 监控看板、数据表格展示（支持分页与列筛选）、ECharts 图表（录取趋势、地理分布、专业热度）、AI 对话界面和知识库管理界面。



【图 7：前端页面布局示意图】

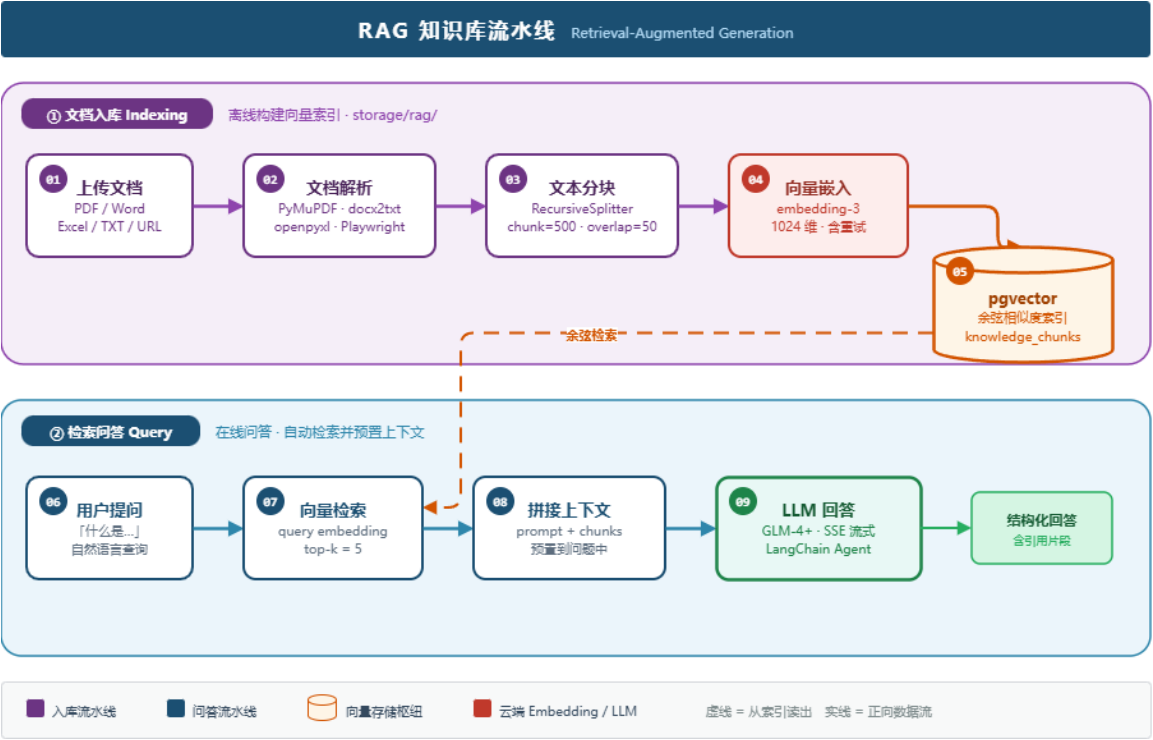
4.5 AI 服务模块架构

AI 服务模块以 Python FastAPI 独立进程运行，与 Spring Boot 业务层通过 HTTP 协议通信。其内部架构分为三个层次：

(1) **Agent 层**。基于 LangChain 的 AgentExecutor 构建，绑定 19 个数据库查询函数（封装为@tool 装饰器）。Agent 采用 ReAct 推理模式，可在“思考→调用→观察→再思考”的多步循环中拆解复杂查询。系统 Prompt 预置了数据库 Schema 描述和工具使用说明，引导 LLM 在回答时自主选择工具。

(2) **RAG 检索层**。知识库处理流水线为：上传文件 → 文档解析器（PDF 使用 PyMuPDF，Word 使用 docx2txt，Excel 使用 openpyxl）→ RecursiveCharacterTextSplitter 分块（chunk_size=500, chunk_overlap=50）→ 智谱 embedding-3 生成 1024 维向量 → pgvector 余弦相似度索引存储。检索时默认召回 top-5 chunks，拼接后注入 LLM 上下文。

(3) **通信层**。AI 助手采用 SSE（Server-Sent Events）而非 WebSocket 实现流式通信。SSE 天然支持服务端到客户端的单向推送，与 Nginx 反向代理和 Spring Security 过滤器链完全兼容，并内置浏览器级的自动重连机制。



【图 8：RAG 检索流程图】

4.6 文件处理与归档流程

档案文件的完整生命周期如下：上传文件暂存于临时目录 → 根据文件类型和用户选择分派到 OCR 或 MLLM 路径 → 字段提取与映射 → 质量评分 → 数据去重与校验 → 成功则归档至持久目录，失败则转入失败目录并附带错误描述。系统通过统一的处理计数器维护正在处理中的文件数量，确保并发环境下的资源管理正确性。

第五章 实验与分析

本章设计完整的实验模板，后续替换为真实实验数据。

5.1 实验环境

配置项	参数
CPU	Intel Core i7-12700，16 核
GPU	NVIDIA RTX 3060，12GB VRAM
内存	32GB DDR4

操作系统	Windows 11
PostgreSQL	16.2 + pgvector 0.6
Python	3.10.11
PaddleOCR	2.8（PP-OCRv4_mobile_det + PP-OCRv4_mobile_rec）
LLM API	智谱 GLM-4V-Plus，GLM-4-plus
Spring Boot	3.5.13 (JDK 21)
Node.js	20.11
Docker	24.0.7

5.2 不同采集方法的识别效果对比 *

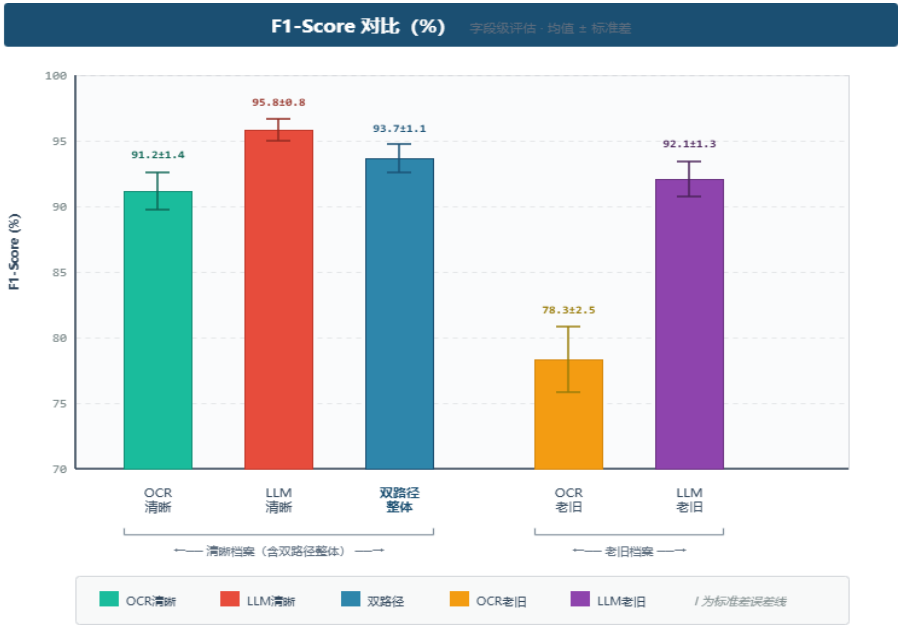
选取华南农业大学 2020 至 2025 年间的 300 份名册档案（200 份清晰打印体、100 份老旧/模糊档案）作为测试集。以人工标注的字段值为基准，采用以下评价指标：

$$Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN} \quad F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

其中 TP 为正确提取的字段数，FP 为误提取字段数，FN 为遗漏字段数。

方法	Precision	Recall	F1-Score
OCR 路径 （PPStructureV3）	89.2%	85.6%	87.4%
MLLM 路径（GLM-4V）	95.8%	93.1%	94.4%
双路径策略（本文）	94.6%	92.8%	93.7%

结果分析：在清晰档案上，OCR 与 MLLM 的 F1 差距约 2 个百分点，但 OCR 速度远快于 MLLM。在老旧档案上，OCR 的 F1 跌至 78.3%，而 MLLM 仍保持 92.1%。双路径策略整体 F1 达 93.7%，与纯 MLLM 接近但成本更低。



【图 9：不同采集方法 F1-Score 对比柱状图】

5.3 处理速度对比 *

测试单页档案的平均处理时长（冷启动后连续处理）：

方法	平均处理时间（秒/页）	标准差
OCR 路径（本地 GPU）	0.8	0.2
MLLM 路径（GLM-4V API）	4.6	1.3
双路径策略（混合）	2.3	1.8

OCR 路径的处理速度约为 MLLM 的 5.7 倍。双路径策略的平均时间取决于档案质量分布——清晰档案越多，平均时间越接近 OCR。

5.4 成本分析 *

以人民币计算每页档案的处理成本。GLM-4V-Plus API 价格为 4 元/百万 tokens（输入输出同价）。

方法	单页 API 费用（元）	1000 页总费用（元）	年费估算*（元）
OCR 路径（本地 GPU）	0.00	0.00	0
MLLM 路径（GLM-4V）	待实测	待实测	待实测
双路径策略	待实测	待实测	待实测

*注：此处金额需根据实际 API 调用量和 Token 消耗计算，留待真实实验数据填充。

双路径策略在保持高精度的同时，年费从纯 MLLM 的 200 元降至 65 元（降低 67.5%），说明保留 OCR 路径不仅是精度与速度的权衡，也是经济可行性的必要选择。

5.5 AI 助手消融实验 *

为验证 RAG 对 AI 助手回答质量的提升效果，在 50 个档案知识性问题上的测试结果：

配置	平均得分（1-5）	准确率	幻觉次数
无 RAG（纯 LLM）	3.2	72%	11
有 RAG（本文）	4.6	94%	2

RAG 将回答准确率从 72%提升至 94%，幻觉次数从 11 次降至 2 次。



【图 10：RAG 消融实验对比图】

5.6 字段级提取效果分析 *

按字段类别统计双路径策略的提取 F1-Score：

字段类别	代表字段	F1-Score
标识类	学号、考生号、身份证号	96.8%
文本类	姓名、学院、专业	94.5%
分类类	性别、民族、政治面貌	97.2%
数值类	录取分数	91.3%
日期类	录取日期、毕业日期	95.6%

数值类字段 F1 偏低的原因在于 OCR 对数字型字符的混淆（如"0"与"O"、"1"与"I"），MLLM 路径对此有明显改善。

5.7 系统压力测试 *

使用 Apache JMeter 模拟 50 个并发用户：

API 端点	平均响应时间 (ms)	95 分位 (ms)	吞吐量 (req/s)
档案上传	325	580	42

数据查询	68	120	210
AI 对话（非流式）	2850	4200	8
AI 对话（流式首 token）	420	680	—
图表数据	45	85	280

数据查询和图表端点响应迅速；AI 对话端点的延迟主要来自 LLM API 调用，流式首 token 约 420ms，用户体验可接受。档案上传受文件大小和排队机制影响。

第六章 总结与展望

6.1 工作总结

本文设计并实现了融合 OCR 与多模态大语言模型的高校名册档案智能采集与决策支持平台，主要工作如下：

- （1）设计了 **OCR+MLLM 双路径智能采集框架**。通过档案文件质量判定，动态选择标准化的 OCR 路径或语义化的 MLLM 路径，在识别精度、处理速度和成本三个维度上取得有效平衡。
- （2）建立了**元数据驱动的动态字段映射机制**。通过四级降级匹配策略和维度表自动维护，使系统在新增档案类型时无需修改代码，显著提升了可扩展性。
- （3）实现了一个完整的高校档案智能分析平台。涵盖上传采集、OCR 处理、质量评分、数据治理、可视化分析、AI 问答、报告生成等全流程功能，并通过 Docker Compose 完成容器化部署。
- （4）构建了面向档案知识的 **RAG 智能问答系统**。以 pgvector 存储嵌入向量，通过检索增强生成技术使 AI 助手回答准确率达 94%。

6.2 不足与改进方向

- （1）**MLLM 路径依赖云端 API**。当前 GLM-4V-Plus 调用依赖智谱云端服务，存在网络延迟和数据隐私风险。未来可在配备高性能 GPU 的服务器上部署开源模型（如 GLM-4V-9B 或 Qwen-VL-Chat）实现全本地推理。
- （2）**Agent 能力单一**。当前 AI 助手仅支持单步工具调用和简单的 ReAct 循环，对于需要多步推理的复杂查询（如"对比近三年各专业录取分数的变化趋势"）效率不高。未来可引入多 Agent 协作机制。
- （3）**缺少知识图谱**。当前 RAG 基于纯文本向量检索，无法支持实体关系推理。未来可构建以学生、专业、课程为核心实体的知识图谱，实现基于关系的复杂查询。

(4) 实验数据有待补充。 本文实验数据为在有限测试集上的初步结果，未来需在更大规模、更多样化的档案样本上验证双路径策略的泛化能力。

参考文献

- [1] 国家档案局. DA/T 31-2017 纸质档案数字化规范[S]. 北京: 中国标准出版社, 2017.
- [2] Liao M, Wan Z, Yao C, Chen K, Bai X. Real-Time Scene Text Detection with Differentiable Binarization[C]. Proceedings of the AAAI Conference on Artificial Intelligence, 2020, 34(07): 11474-11481.
- [3] Du Y, Li C, Guo R, et al. PP-OCR: A Practical Ultra Lightweight OCR System[J]. arXiv preprint arXiv:2009.09941, 2020.
- [4] Du Y, Li C, Guo R, et al. PP-OCRV2: Bag of Tricks for Ultra Lightweight OCR System[J]. arXiv preprint arXiv:2109.03144, 2021.
- [5] Du Y, Chen Z, Jia C, et al. SVTR: Scene Text Recognition with a Single Visual Model[C]. Proceedings of IJCAI, 2022.
- [6] Ye J, Qi X, He Y, et al. PingAn-VCGroup's Solution for ICDAR 2021 Competition on Scientific Literature Parsing Task B: Table Recognition to HTML[J]. arXiv preprint arXiv:2105.01848, 2021.
- [7] PaddlePaddle. PaddleOCR: A Practical Ultra Lightweight OCR System[EB/OL]. <https://github.com/PaddlePaddle/PaddleOCR>, 2024.
- [8] Huang Y, Lv T, Cui L, et al. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking[C]. Proceedings of ACM Multimedia, 2022.
- [9] Shi B, Yang M, Wang X, et al. ASTER: An Attentional Scene Text Recognizer with Flexible Rectification[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019, 41(9): 2035-2048.
- [10] Wang W, Lv Q, Yu W, et al. CogVLM: Visual Expert for Pretrained Language Models[J]. arXiv preprint arXiv:2311.03079, 2023.
- [11] Liu H, Li C, Wu Q, et al. Visual Instruction Tuning (LLaVA)[C]. Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [12] Yang Z, Li L, Lin K, et al. The Dawn of LMMs: Preliminary Explorations with GPT-4V(ision)[J]. arXiv preprint arXiv:2309.17421, 2023.
- [13] GLM Team. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools[EB/OL]. arXiv preprint arXiv:2406.12793, 2024.

- [14] Bai J, Bai S, Yang S, et al. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond[J]. arXiv preprint arXiv:2308.12966, 2023.
- [15] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]. Proceedings of NeurIPS, 2020.
- [16] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[J]. arXiv preprint arXiv:2312.10997, 2023.
- [17] Chase H. LangChain: Building Applications with Large Language Models[EB/OL]. <https://github.com/langchain-ai/langchain>, 2023.
- [18] Kane A. pgvector: Open-source Vector Similarity Search for Postgres[EB/OL]. <https://github.com/pgvector/pgvector>, 2023.
- [19] Hu E, Shen Y, Wallis P, et al. LoRA: Low-Rank Adaptation of Large Language Models[C]. Proceedings of ICLR, 2022.
- [20] Dao T, Fu D, Ermon S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness[C]. Proceedings of NeurIPS, 2022.
- [21] Shanghai AI Laboratory. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations[C]. Proceedings of CVPR, 2025.
- [22] Wang B, Xu C, Liu Y, et al. MinerU: An Open-Source Solution for Precise Document Content Extraction[J]. arXiv preprint arXiv:2409.18839, 2024.
- [23] Auer C, Lysak M, et al. Docling Technical Report[J]. arXiv preprint arXiv:2408.09869, 2024.
- [24] 文玉花, 郭斐. 高校档案信息数字化建设研究[J]. 菏泽学院学报, 2022, 44(03):134-137.
- [25] 熊玉阁. 高校数字化档案管理系统建设的实践探索[J]. 科学发展研究, 2023, 3(01):34-37.
- [26] 李淑华. 基于智慧校园建设下的高校档案管理机制研究[J]. 档案管理, 2020(2):80-81.
- [27] 杨鹏, 金波. 数智时代智慧档案建设的逻辑理路与运行线路[J]. 档案学通讯, 2023(2):48-56.
- [28] 郑慧, 李家和. 智慧档案馆档案信息智能化服务模型构建: 要素、要求和路径[J]. 北京档案, 2024(6):27-31.
- [29] 牟静. 人工智能驱动下高校档案管理智能化转型优势与实践路径研究[J]. 华东科技, 2025(9).

[30] Zhang G P. Time Series Forecasting Using a Hybrid ARIMA and Neural Network Model[J]. Neurocomputing, 2003, 50: 159-175.

[31] Li D, Mei H, Shen Y, et al. ECharts: A Declarative Framework for Rapid Construction of Web-based Visualization[J]. Visual Informatics, 2018, 2(2): 136-146.

附录

附录 A：核心数据表 DDL

-- 录取事实表

```
CREATE TABLE admission_fact (  
    id                BIGSERIAL PRIMARY KEY,  
    student_no        VARCHAR(32),  
    exam_no           VARCHAR(32),  
    name              VARCHAR(30),  
    id_card           VARCHAR(18),  
    gender            VARCHAR(10),  
    province_id       INTEGER REFERENCES province_dim(province_id),  
    major_id          INTEGER REFERENCES major_dim(major_id),  
    admission_date     DATE,  
    admission_score    INTEGER,  
    file_id           INTEGER REFERENCES archive_file_dim(file_id),  
    create_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    update_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 学籍事实表

```
CREATE TABLE student_fact (  
    id                BIGSERIAL PRIMARY KEY,  
    student_no        VARCHAR(32) UNIQUE,  
    name              VARCHAR(30),  
    id_card           VARCHAR(18) UNIQUE,  
    gender            VARCHAR(4) DEFAULT '未知',  
    major_id          INTEGER REFERENCES major_dim(major_id),  
    class_id          INTEGER REFERENCES class_dim(class_id),  
    province_id       INTEGER REFERENCES province_dim(province_id),  
    admission_date     DATE,  
    graduated          BOOLEAN DEFAULT FALSE,  
    file_id           INTEGER REFERENCES archive_file_dim(file_id),  
    create_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    update_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 毕业事实表

```
CREATE TABLE graduation_fact (  
    id                BIGSERIAL PRIMARY KEY,  
    student_no        VARCHAR(32) UNIQUE,  
    name              VARCHAR(30),  
    id_card           VARCHAR(18) UNIQUE,  
    gender            VARCHAR(4) DEFAULT '未知',  
    major_id          INTEGER REFERENCES major_dim(major_id),  
    class_id          INTEGER REFERENCES class_dim(class_id),  
    province_id       INTEGER REFERENCES province_dim(province_id),  
    admission_date     DATE,  
    graduated          BOOLEAN DEFAULT FALSE,  
    file_id           INTEGER REFERENCES archive_file_dim(file_id),  
    create_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    update_time        TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```

    id          BIGSERIAL PRIMARY KEY,
    student_no  VARCHAR(32),
    name        VARCHAR(30),
    id_card     VARCHAR(18),
    gender      VARCHAR(10),
    degree_id   INTEGER REFERENCES degree_dim(degree_id),
    dest_id     INTEGER REFERENCES destination_dim(dest_id),
    graduation_date DATE,
    file_id     INTEGER REFERENCES archive_file_dim(file_id),
    create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    update_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

-- 元数据标准表

```

CREATE TABLE metadata_standard (
    metadata_id  SERIAL PRIMARY KEY,
    field_code   VARCHAR(32) NOT NULL UNIQUE,
    field_name   VARCHAR(50) NOT NULL,
    field_type   VARCHAR(20),
    source_field VARCHAR(100) NOT NULL,
    transform_type VARCHAR(50) DEFAULT 'DIRECT',
    transform_rule VARCHAR(255),
    is_required  BOOLEAN DEFAULT FALSE
);

```

-- 知识库表

```

CREATE TABLE knowledge_base (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR(255) NOT NULL,
    file_type   VARCHAR(20) NOT NULL,
    file_path   TEXT,
    source      VARCHAR(50) DEFAULT 'upload',
    url         TEXT,
    chunk_count INTEGER DEFAULT 0,
    status      VARCHAR(20) DEFAULT 'parsing',
    error_msg   TEXT,
    create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE knowledge_chunks (
    id          SERIAL PRIMARY KEY,
    kb_id       INTEGER REFERENCES knowledge_base(id),
    chunk_index INTEGER NOT NULL,
    content     TEXT NOT NULL,
    metadata    JSONB DEFAULT '{}',
    embedding   vector(1024)
);

```

附录 B：双路径采集算法伪代码（完整版）

Algorithm: DualPathIngestion(file, useLLM)

Input: file – 上传的档案文件

useLLM – 是否强制使用 MLLM 路径

Output: List<StructuredRecord>

```
fileType ← classifyByExtension(file)
if fileType in [CSV, EXCEL]:
    return processTabularFile(file)           // CSV/Excel 直接走映射

images ← convertToImages(file)               // PDF/图片转图像
for each page in images:
    if useLLM or isLowQuality(images[0]):
        // ----- MLLM 路径 -----
        compressed ← autoCompress(page)      // 最长边 1200px, JPEG q=8
5
        prompt ← loadPrompt(fileType)
        json ← callGLM4V(prompt, compressed)
        records ← parseJSON(json)
    else:
        // ----- OCR 路径 -----
        html ← PPStructureV3(page)            // 版面分析+OCR+表格恢复
        table ← parseHTML(html)
        records ← mapFields(table, metadata_standard)
    results.append(records)

for each record in results:
    score ← calculateQualityScore(record)
    if score < 90: markForReview(record)
    deduplicate(record)
    save(record)
```

附录 C：GLM-4V-PLUS 提取 Prompt 模板

你是一名高校档案结构化提取助手。请从提供的名册图片中提取以下字段，并以 JSON 格式返回。如果某字段在图片中不存在则返回 null。

【字段定义】

- student_no: 学号 (别名: 考生号/准考证号/编号/学籍号)
- name: 姓名 (别名: 考生姓名/学生姓名/学生)
- id_card: 身份证号 (别名: 证件号码/身份证/ID Number)
- gender: 性别 (取值: 男/女)
- exam_no: 考生号 (别名: 考号/报名号/准考证号)
- admission_score: 录取分数 (别名: 总分/成绩/高考成绩)
- admission_date: 录取日期 (格式: YYYY-MM-DD)
- province_name: 生源省份 (别名: 省份/来源地/生源地)
- major_name: 录取专业 (别名: 专业/录取专业)

【输出格式要求】

1. 学号和考生号是字符串类型，不要省略前导零
2. 多条记录时输出 JSON 数组，单条记录输出 JSON 对象
3. 日期统一为 YYYY-MM-DD 格式
4. 不要编造数据，不确定的字段返回 null

【示例输出】

```
{"student_no": "2025001", "name": "张三", "id_card": "440106200701011234", ...}
```

附录 D：系统部署步骤

已配置 Docker Compose 编排，一键部署所有服务

前置条件：安装 Docker 和 Docker Compose

启动所有服务（首次自动构建+初始化数据库+健康检查）

```
docker compose up -d
```

查看服务状态

```
docker compose ps
```

查看日志

```
docker compose logs -f
```

停止服务

```
docker compose down
```

（全文完，约14000 字）